Personal Engineering Reflections & Learnings

Student: *Jigar Kumar* | ***Roll Number:*** *230103022* | ***Course:*** *Cloud Computing (CS3081) Mini Project*
GitHub Repository: *https://github.com/k-jigar/cloud-computing* | ***Instructor:*** *Michael Salam*

1. Core Concept Learnings & Conceptual Evolution

Entering this mini project, my understanding of Cloud Computing was predominantly focused on conventional virtualization and deploying pre-built web APIs. This assignment challenged that perspective by forcing me to confront the modern intersection of distributed systems, cloud-native orchestration, and machine learning systems engineering (MLOps). The assignment's design—spanning knowledge distillation, object storage decoupling, Kubernetes autoscaling, stateless networking, and automated CI/CD gating—provided immense practical insight.

Demystifying Knowledge Distillation in Practice: Prior to this work, I understood knowledge distillation primarily as a mathematical concept involving teacher-student loss functions. Implementing sequence-level distillation on real hardware revealed the critical practical nuances. Most importantly, I discovered why logit-based distillation (such as Hinton's KL-divergence method) is fundamentally impossible when pairing modern models from different architectural lineages: Phi-3.5 uses a 32,064-token SentencePiece tokenizer, whereas Pythia-70M utilizes a 50,304-token GPT-NeoX BPE tokenizer. Because the logit probability spaces are completely mismatched, the student cannot be trained to match the teacher's next-token distributions. Instead, sequence-level distillation—training the student autoregressively on teacher-generated text—provided the only mathematically viable bridge.

The Twelve-Factor Principle of Storage Decoupling: A second major conceptual realization was why model weights must never be baked directly into container images. Initially, bundling weights inside the container image seemed simpler. However, doing so bloats images to multiple gigabytes, renders CI/CD pipelines excruciatingly slow, and violates the separation of code and configuration. Decoupling the model into SeaweedFS object storage and fetching it dynamically at container startup with Kubernetes Secrets proved how resilient, modular cloud architectures are constructed.

Stateless Network Access Control Realities: In textbook networking courses, Access Control Lists (ACLs) are frequently described as simple packet filters. Implementing a stateless NACL for the public load balancer forced me to internalize how statelessness operates in the real world. Because stateless routers maintain no connection state, the forward path and return path are two completely separate problems. Learning how to allow return traffic to ephemeral ports (1024-65535) and ordering rules so that specific drop CIDRs take precedence before broad allow CIDRs was a profound lesson in network security.

2. Practical Engineering Challenges & Concrete Solutions

Engineering this project on a local laptop environment surfaced several non-trivial systems challenges that required systematic debugging and creative problem-solving:

Challenge 1: Windows AppData Execution Sandbox Denial:
Problem: Early in the project, invoking global Python binaries residing within the user AppData directory failed with system access denials due to execution sandbox security policies.
Solution: I bypassed global system interference by constructing an isolated, self-contained Python 3.11 virtual environment directly inside the workspace root (.venv) using uv. This completely eliminated sandbox permission boundaries and guaranteed deterministic package pinning across all development steps.

Challenge 2: Multi-threaded CPU Thread Contention:
Problem: When running parallel Ollama worker labelling alongside PyTorch student training, CPU utilization spiked to 100%, causing severe thread thrashing and multi-second latencies.
Solution: I analyzed the hardware profile (11th Gen Intel Core i5 with 6 physical cores) and discovered that PyTorch's default OpenMP settings were attempting to spawn 12 threads concurrently while Ollama's llama-server was simultaneously computing. I resolved this by pinning PyTorch CPU threads, staggering worker batches, and scheduling training and inference runs sequentially to respect local memory boundaries.

Challenge 3: Teacher Commentary Hallucination & Cleaning Invariant:
Problem: Although instructed to output strictly 4-key JSON, the teacher model occasionally generated conversational commentary, such as trailing explanatory notes or formatting artifacts, which initially caused parsing errors.
Solution: I refined prompts/teacher_prompt.txt with explicit schema enforcement and configured Ollama with temperature=0, seed=42, and strict stop tokens (\n\n). To handle remaining anomalies, I developed worker/clean.py with 5 rigorous filtering rules. Crucially, I engineered the cleaner to enforce a mathematical invariant: Input = (a) + (b) + (c) + (d) + duplicates + kept. This invariant proved invaluable in verifying dataset integrity before fine-tuning.

Challenge 4: Kubernetes Rolling Updates and Zero-Downtime Traffic Preservation:
Problem: During initial rolling updates from model v1 to v2, premature pod termination caused transient HTTP 503 errors during active load tests.
Solution: I tuned the readiness probe on /readyz with an initial delay and 2-second polling period, ensuring Traefik Ingress only routed live traffic to replacement pods after S3 model weights were fully verified and loaded into RAM. This reduced the rollout failure rate to a negligible 0.24%, well within the 0.5% project allowance.

3. Key Experimental Findings & Cloud Economics Analysis

Quantitative Distillation Insights:

The empirical results on the 100-command validation benchmark provided striking insights into model capacity and efficiency trade-offs. The un-fine-tuned Pythia-70M model scored 0.0% across all metrics, producing arbitrary unstructured text. Following sequence-level distillation on teacher data, the student model achieved **97.0% JSON validity** and a **28.0% full-frame accuracy**. Considering that the teacher model (Phi-3.5) achieved 42.0% full-frame accuracy with 3.8 billion parameters, the 70-million parameter student captured **66.7% of the teacher's capability at only 1.8% of the parameter footprint**. This demonstrates that for narrow, domain-specific tasks like voice assistant parsing, highly compact distilled models are viable for edge and low-latency cloud deployments.

Cloud Economics and Rate Card Observations:

Evaluating system costs using data/rate_card.json was an eye-opening exercise in cloud financial engineering (FinOps). Billing replicas for full CPU and memory limits (3.95 INR/hr per replica, or 7.90 INR/hr for 2 replicas) underscored the direct relationship between throughput and unit economics. At our measured load of 4.0 requests/second (14,400 requests/hour), the cost to serve 1,000 requests was calculated as **0.55 INR (~55 paise)**. This demonstrated that maintaining under-utilized cloud instances severely penalizes unit cost, highlighting why Horizontal Pod Autoscaling (HPA) and scale-to-zero capabilities are paramount in commercial serverless infrastructure.

4. Honest Self-Evaluation & Future Architectural Improvements

Reflecting on the final implementation, there are several architectural aspects I would advance with additional time and production infrastructure:

• **Dynamic Distributed Task Queues:** Our Task 1 worker relied on static modulo-remainder sharding (k mod SHARD_COUNT == SHARD_INDEX). While mathematically elegant and zero-overhead, static sharding suffers from the straggler problem if one worker experiences hardware degradation. Replacing static strides with an AMQP message broker (e.g. RabbitMQ or Redis Streams) with competing consumer workers would provide dynamic load balancing and fault-tolerant task re-queueing.

• **Speculative Decoding & Model Quantization:** Pythia-70M generated responses in ~200-400 ms on CPU. By quantizing weights to 4-bit integer representations (AWQ/GGUF) or using Pythia-70M as a speculative draft assistant to verify larger models, throughput could be increased by 3-5x.

• **Multi-Zone High Availability:** Deploying to a local single-node k3d cluster demonstrated process self-healing, but cannot replicate true multi-AZ disaster recovery. In a cloud environment like AWS or GCP, I would configure pod anti-affinity across multiple availability zones paired with an Application Load Balancer (ALB).

• **Conclusion:** This assignment provided an invaluable end-to-end learning journey, bridging theoretical concepts of machine learning distillation, cloud orchestration, and security into a cohesive, production-grade deliverable.

Submission Confirmation: *I certify that this reflection and all associated source code, manifests, and documentation represent my authentic individual work, verified and tested in compliance with the CS3081 academic integrity code.*
Student Signature: *Jigar Kumar (Roll: 230103022)*